

```

# Motorized Screens PR Implementation Proof
## 1) Exact TypeScript type definitions from 'packages/domain/src/motorizedScreens.ts'
'''ts
export const MODULE_KEY = 'motorized_screens_ordering_tool';
export type ScreenManufacturer = 'DOS Screens' | 'MagnaTrack';
export type MotorType = 'Alpha';
export type RawMeasurements = {
  UL: number;
  LL: number;
  OL: number;
  UR: number;
  LR: number;
  OR: number;
  T: number;
  M: number;
  B: number;
};
export type ScreenConfigSection = {
  screenLabel: string;
  frameColor?: string;
  frameType?: string;
};
export type MotorConfigSection = {
  motorType?: MotorType;
  controlType?: string;
  powerNotes?: string;
};
export type OrderMeasurementsSection = {
  orderedWidthIn: number;
  orderedHeightIn: number;
};
export type MaterialsSection = {
  meshType?: string;
  splineColor?: string;
};
export type MiscSection = {
  notes?: string;
};
export type AttachmentsSection = {
  attachmentIds: string[];
};
export type MotorizedScreenItemInput = {
  screenNumber: number;
  manufacturer: ScreenManufacturer;
  section1: ScreenConfigSection;
  section2: MotorConfigSection;
  section3: OrderMeasurementsSection;
  section4: MaterialsSection;
  section5: MiscSection;
  section6Raw: RawMeasurements;
  section7: AttachmentsSection;
};
export type DerivedMeasurementSummary = {
  upperSlope: number;
  slopeDirection: 'left_high' | 'right_high' | 'flat';
  highSide: number;
  lowSide: number;
  tbDiff: number;
};
export function validateRawMeasurements(raw: RawMeasurements): string[] {
  const errors: string[] = [];
  (Object.entries(raw) as Array<[keyof RawMeasurements, number]>).forEach(([key, value]) => {
    if (typeof value !== 'number' || Number.isNaN(value) || value <= 0) errors.push(`${key} must be a positive number`);
  });
  return errors;
}
export function validateScreenItem(item: MotorizedScreenItemInput): string[] {
  const errors: string[] = [];
  if (item.screenNumber < 1 || !Number.isInteger(item.screenNumber)) errors.push('screenNumber

```